

DSA STUDY BLUEPRINT SERIES

34. Subarray Sum Equals K

Prefix Sum Cumulative Map Counting

Section 05 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Subarray Sum:** Subarray sum between index i and j equals `prefix[j] - prefix[i-1]`.
- Match using formula: `runningSum - target` prefix seen counts.

Memory & Execution Blueprint

Running Sum Maps checks (Target = 3):

Curr Sum = 6 • Search $6 - 3 = 3$ in map $\Rightarrow$ If 3 exists, increment total count!

C++ Implementation Reference

Standard code layout and syntax

```
int subarraySum(vector& nums, int k) {  
    unordered_map m;  
    m[0] = 1; // Base condition  
    int sum = 0, count = 0;  
    for (int x : nums) {  
        sum += x;  
        if (m.count(sum - k)) count += m[sum - k];  
        m[sum]++;  
    }  
    return count;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** This approach handles negative array values correctly, unlike the standard sliding window.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace check complement lookups in a hash map for `Two Sum` on `[2, 7, 11]` (target = 9):

Element Scan	Complement: <code>target - x</code>	Hash Map Search	Map Insertion
2	$9 - 2 = 7$	Not found	Insert 2 mapping index 0
7	$9 - 7 = 2$	Found 2 (at index 0!)	Return indices [0, 1]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Prefix Sum Math:** Resolve sub-array queries using math transformations. Complement hash lookups count occurrences instantly.
- **Hash Collisions:** Handled via separate chaining (lists link) or open addressing (linear index probing).
- **Related LeetCode Problems:** #560 Subarray Sum Equals K, #1 Two Sum, #49 Group Anagrams.